2026

Artificial Intelligence 5001 : Assignment 1

Contents

Introduction 1
Sudoku Performance 1
Supervised Learning: Regression 2
Agent Framework 2
Regression Performance 2
Reflection & Comparison 3
Tables & Graphics 4
Appendices 27
Appendix Part 1 27
Appendix Part 2 34
References 41

Introduction

This portfolio examines the implementation and performance evaluation of search Algorithms and supervised learning for AI agent techniques. The first part will analyse the application of Breadth-First Search and Depth-First Search to Sudoku boards of various sizes (4x4, 6x6, and 9x9) and difficulties (Easy, Medium, Hard, and Very Hard), with three solved instances for all combinations unsolved versions displayed in **Figure 12A**. The complexity of each Algorithm was analysed using tracemalloc and time which was then tabularized and transformed.

The second part will discuss supervised learning within the agent framework, identifying agent-type, environment, sensors, actuators, and efficiency, before examining regression performance on a house-price training prediction dataset. Finally, providing a comparative reflection and discussion of the previously mentioned search and supervised learning algorithms, highlighting strengths, weaknesses, similarities, and differences.

Sudoku Performance

The Depth-First Search (DFS) algorithm performed poorly. It used more memory and had longer runtimes, as shown in **Figure 1A and Figure 2B.** DFS often explores fewer total states than Breadth-First Search (BFS). DFS only surpassed BFS on the hardest boards, where BFS must traverse millions of states. This is surprising because DFS is the standard method for Sudoku. DFS usually solves puzzles quickly by filling one board fully before backtracking if it is invalid. BFS explores all options in a single cell before moving to the next, delaying full board completion. The inefficiency in DFS here stems from repeatedly packing and unpacking tuples, which adds computational overhead.

DFS has space complexity close to O(n log n), as supported by **Figure 1A**, and exhibits nearly linear time complexity. BFS, conversely, exhibits O(n log n) time and space as seen in **Figure 1A and Figure 2B**. This explains why a less-optimized DFS can still surpass BFS on high-complexity boards; BFS takes up to 38.9% longer to solve such instances. Both algorithms are complete and sound: they guarantee a solution if one exists and correctly report insolvability. If multiple solutions exist, only the first instance is returned. BFS is cost-optimal in that it finds the shallowest solution, while DFS may traverse deeper, costlier branches before identifying a shallower solution.

Supervised Learning: Regression

Agent Framework

Regression, a type of continuous learning agent in the Agent Framework, predicts patterns from environmental data. It uses a dataset based on Californian housing prices. Features include longitude, latitude, proximity to the ocean, median house value, and more. This allows for a thorough analysis of price factors. The agent's sensors extract data from a CSV file. They prepare the data for analysis by handling missing values and normalizing the data. The data is split into training and test sets to evaluate the model. During training, the agent uses actuators to update its model. It learns from the training data to improve predictions and reduce errors. Once trained, the agent uses actuators to predict new house values. Performance is measured by R^2, showing how well the model explains the data, and by Mean Absolute Percentage Error (MAPE), which gives the average prediction error as a percentage. These metrics help judge the model's accuracy and guide future improvements.

Regression Performance

R^2 provides a primary measure of regression model performance, capturing the explained variance in house prices (Geeksforgeeks, 2025). Linear Regression attained an R^2 of 0.65, indicating moderate predictive strength but limited ability to capture nonlinearities. In contrast, Random Forest Regression reached an R^2 of 0.83, reflecting improved modelling of complex curves. MAPE, used alongside R^2, expresses the mean prediction error as a percentage of actual prices; both models achieved a MAPE of 28.60%, indicating a mean deviation of ~29%. Lower MAPE values signify more accurate models (Geeksforgeeks, 2021). Overfitting or underfitting can be inferred by comparing the training and test R^2 values. Linear Regression's 0.003% overfit is negligible, while Random Forest's 0.15% underfit may warrant further tuning, though this can introduce computational complexity. We can visualize the prediction accuracy using a scatter plot of predicted vs. actual values, where the model performs quite well until the end, after which there is a random spike of inaccurate predictions. A disadvantage of Random Forest Regression is that, due to the multiple decision trees, it can be harder to interpret. (Geeksforgeeks, 2025)

To assess regression model performance, a heatmap was used to visualize the direction and magnitude of each factor's effect on the target variable. In this heatmap, negative values represent factors associated with decreases in the target, while positive values represent increases. These relationships are shown in **Figure 9A**. The implemented regression algorithm predicts property prices beyond the test or training datasets when provided with the relevant feature set displayed in **Figure 11A**.

The utilised dataset included longitudinal and latitudinal coordinates of Californian property censused in the 1990s, a scatter graph can be utilised to represent the borders of California as seen in **Figure 10A**

Reflection & Comparison

There are numerous fundamental differences between uninformed searches such as DFS and BFS and supervised learning techniques such as regression, DFS and BFS are utilised within a pre-defined environment and do not require tuning or training always producing the same output for a given input. On the other hand, regression is a data-driven algorithm that must be tuned and trained with persistent unavoidable errors and uncertainty whereas, DFS and BFS either fail or discover a path the process efficiency can be measured in states, seconds or memory whereas the efficiency of regression is evaluated using statistical metrics such as R_2 and Mean Absolute Error.

Tables & Graphics

Figure 1A

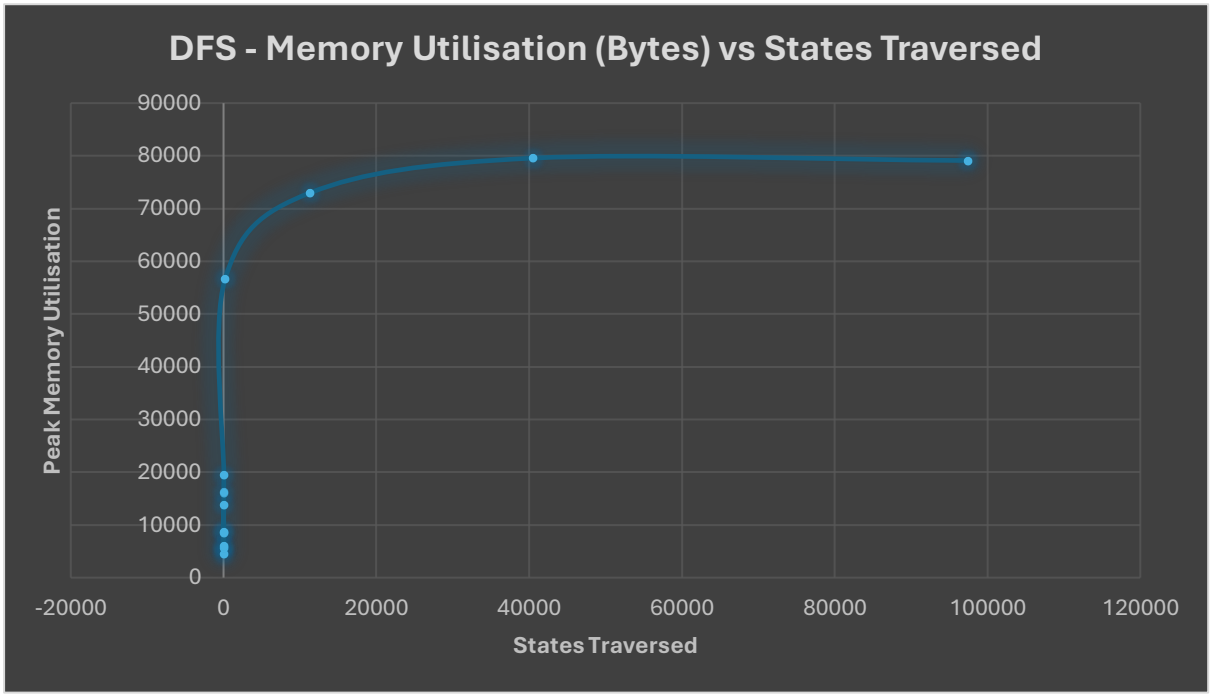

Figure 1B

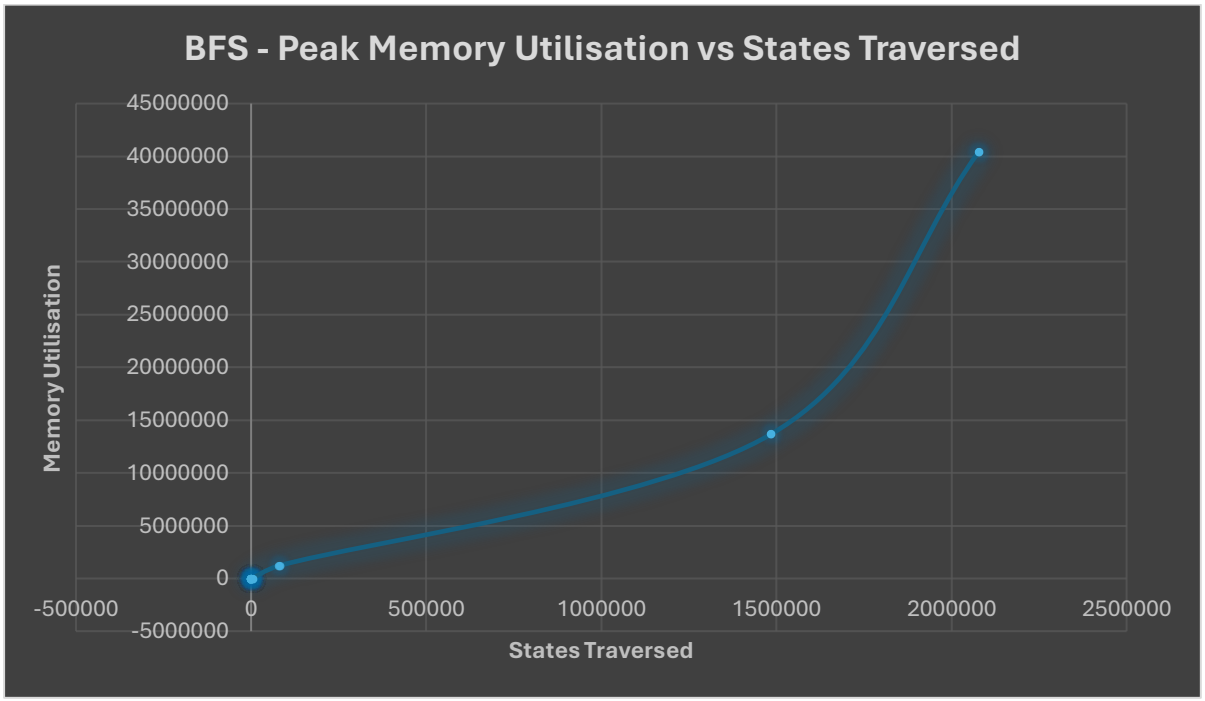

Figure 2A

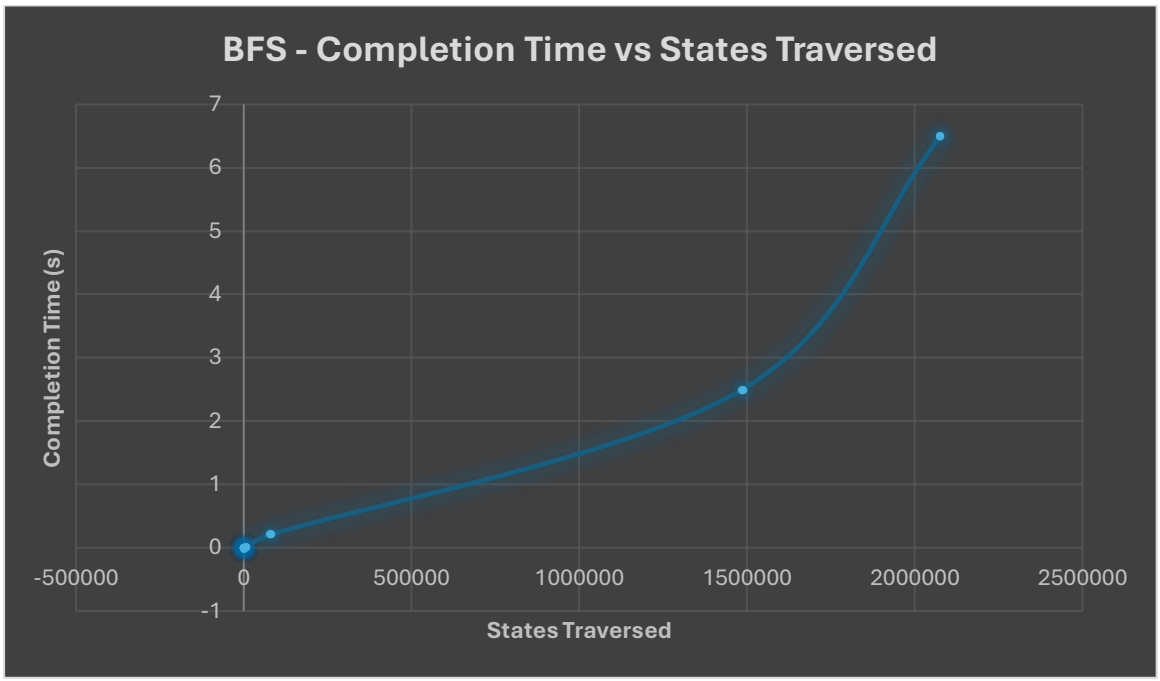

Figure 2B

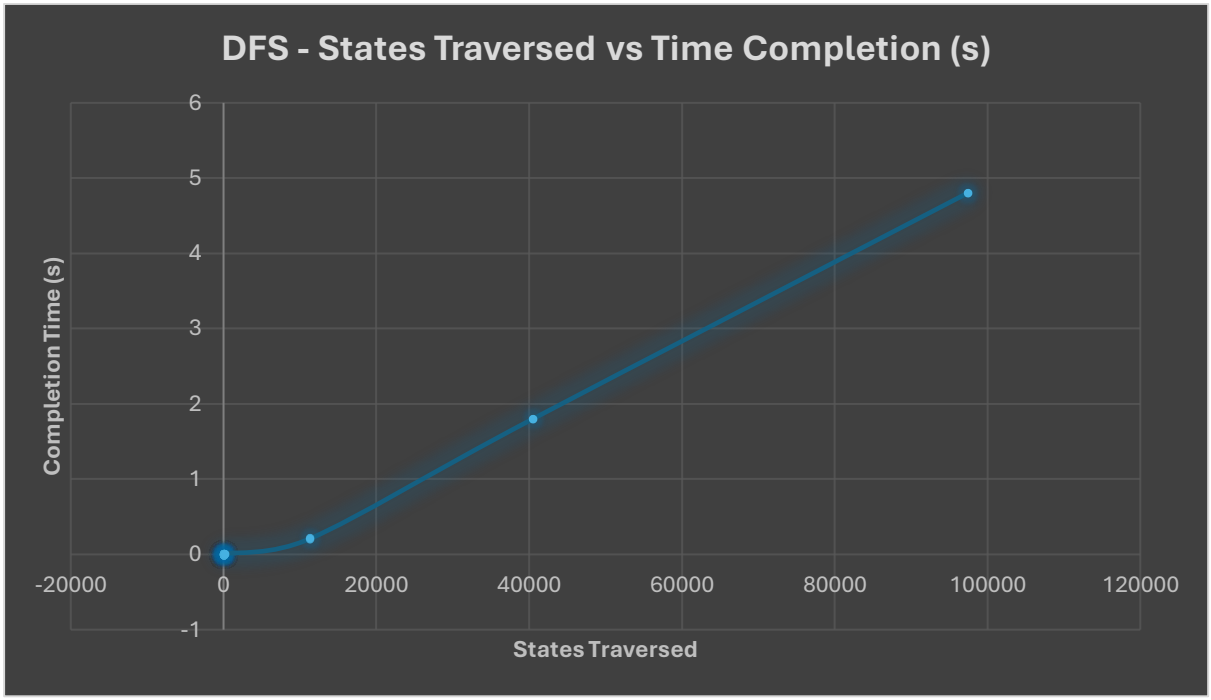

Figure 3A (Space Complexity)

			Breadth Search				Depth Search			
	Board Size		Easy	Medium	Hard	Very Hard	Easy	Medium	Hard	Very Hard
Average Peak Memory (Bytes) 2.dp Average	4x4		1370.67	1698.67	1698.67	2712	4549.33	8728	5576	6016
	6x6		2717.33	2504	5893.33	16280	8480	13797.33	16208	19450.67
	9x9		113653	1207152	40374658.67	13691280	56704	73008	79114.67	79578.67

			Breadth Search				Depth Search			
	Board Size		Easy	Medium	Hard	Very Hard	Easy	Medium	Hard	Very Hard
Peak Memory Consumption (Bytes)	4x4	A	1136	1168	1928	3280	4696	5136	5576	6016
		B	1488	1488	1608	1576	4696	5136	5576	6016
		C	1488	2440	1560	3280	4256	5136	5576	6016
	6x6	A	2400	2504	6896	27344	8480	13808	16208	19440
		B	3992	2504	3888	14496	9224	13808	16208	19472
		C	1760	2504	6896	7000	8480	13776	16208	19440
	9x9	A	87520	318688	8746280	8506064	58096	72080	76320	80496
		B	33960	406104	104954984	13691280	56704	72080	80496	77712
		C	219480	2896664	7422712	37158080	58032	74864	80528	80528

Figure 4A (State Traversal)

			Breadth Search				Depth Search			
	Board Size		Easy	Medium	Hard	Very Hard	Easy	Medium	Hard	Very Hard
Average States Traversed 2.dp	4x4		37.33	53.33	62.67	114.67	9.33	11	11.67	22.67
	6x6		104	118	332	762	10.33	19.33	25	40.33
	9x9		4734	80334	2076876	1485897	215.67	11345	97378.33	40482.33

			Breadth Search				Depth Search			
	Board Size		Easy	Medium	Hard	Very Hard	Easy	Medium	Hard	Very Hard
Total States Traversed	4x4	A	36	40	64	124	9	10	11	29
		B	40	44	72	60	10	11	11	12
		C	36	76	52	160	9	12	13	27
	6x6	A	66	108	372	1086	10	18	21	52
		B	186	126	234	774	11	20	20	43
		C	60	120	390	426	10	20	34	26
	9x9	A	2799	31824	306207	618651	158	2988	21266	32274
		B	2925	26460	5270328	759447	255	1579	229601	57999
		C	8478	182718	654093	3079593	234	6778	41268	31174

Figure 5A (Time Complexity)

			Breadth Search				Depth Search			
	Board Size		Easy	Medium	Hard	Very Hard	Easy	Medium	Hard	Very Hard
Average Completion Time (Seconds 2.sf)	4x4		0.0002	0.00023	0.00029	0.00044	0.00024	0.00025	0.00026	0.00044
	6x6		0.00034	0.00033	0.00094	0.0021	0.00031	0.00052	0.00078	0.0011
	9x9		0.012	0.21	6.5	2.5	0.0098	0.21	4.8	1.8
Average										

			Breadth Search				Depth Search			
	Board Size		Easy	Medium	Hard	Very Hard	Easy	Medium	Hard	Very Hard
Completion Time (Seconds)	**4x4**	A	0.000194	0.000190	0.000383	0.000450	0.000232	0.000245	0.000267	0.000539
		B	0.000197	0.000213	0.000272	0.000261	0.000238	0.000243	0.000240	0.000282
		C	0.000197	0.000294	0.000223	0.000607	0.000238	0.000275	0.000277	0.000512
	6x6	A	0.000253	0.000327	0.001057	0.002822	0.000305	0.000455	0.000655	0.001435
		B	0.000546	0.000402	0.000685	0.002271	0.000318	0.000569	0.000551	0.001247
		C	0.000213	0.000266	0.001064	0.001270	0.000308	0.000536	0.001126	0.000753
	9x9	A	0.006769	0.080925	0.771152	1.5888084	0.006620	0.146366	0.987014	1.378673
		B	0.007220	0.066208	17.003460	2.007411	0.013437	0.069663	10.906972	3.115551
		C	0.020946	0.473089	1.768509	3.981152	0.009378	0.419729	2.436551	0.874545

Figure 6A

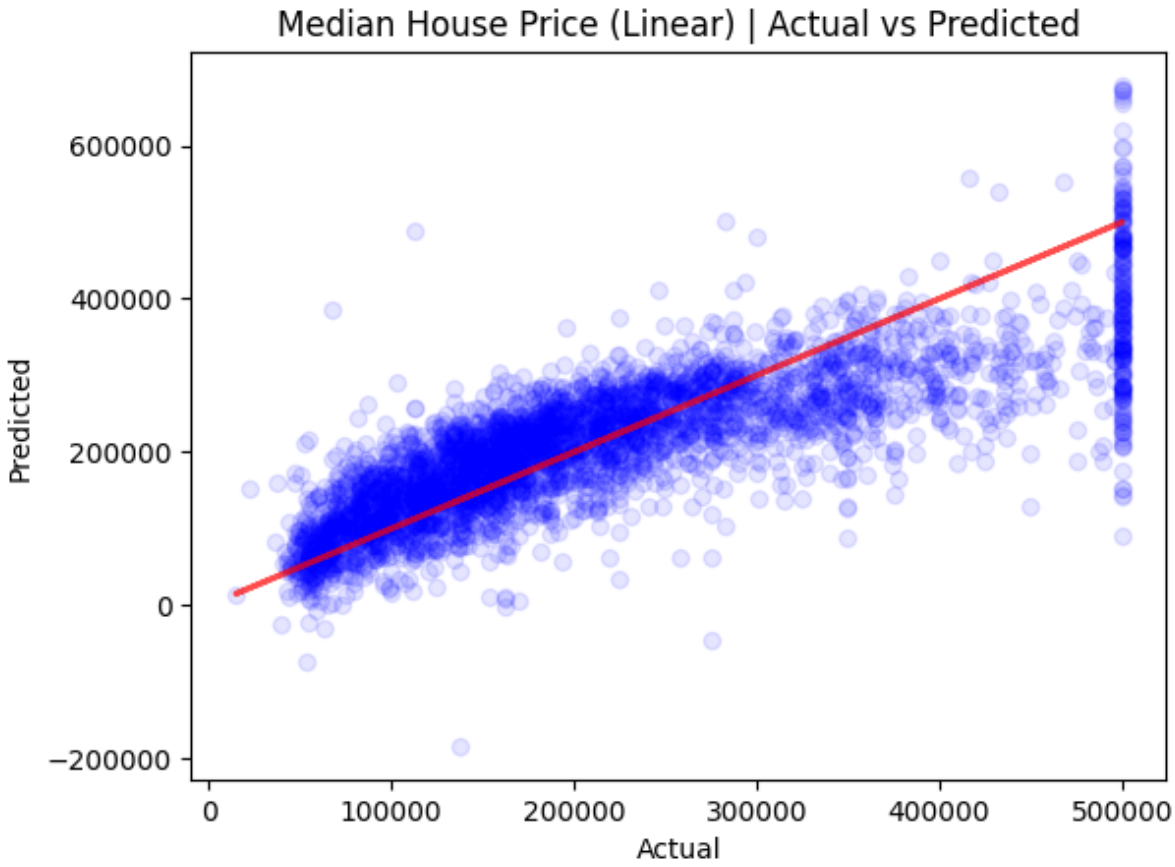
Median House Price (Linear) | Actual vs Predicted
Predicted
600000
400000
200000
0
−200000
0
100000
200000
300000
400000
500000
Actual

Figure 7A

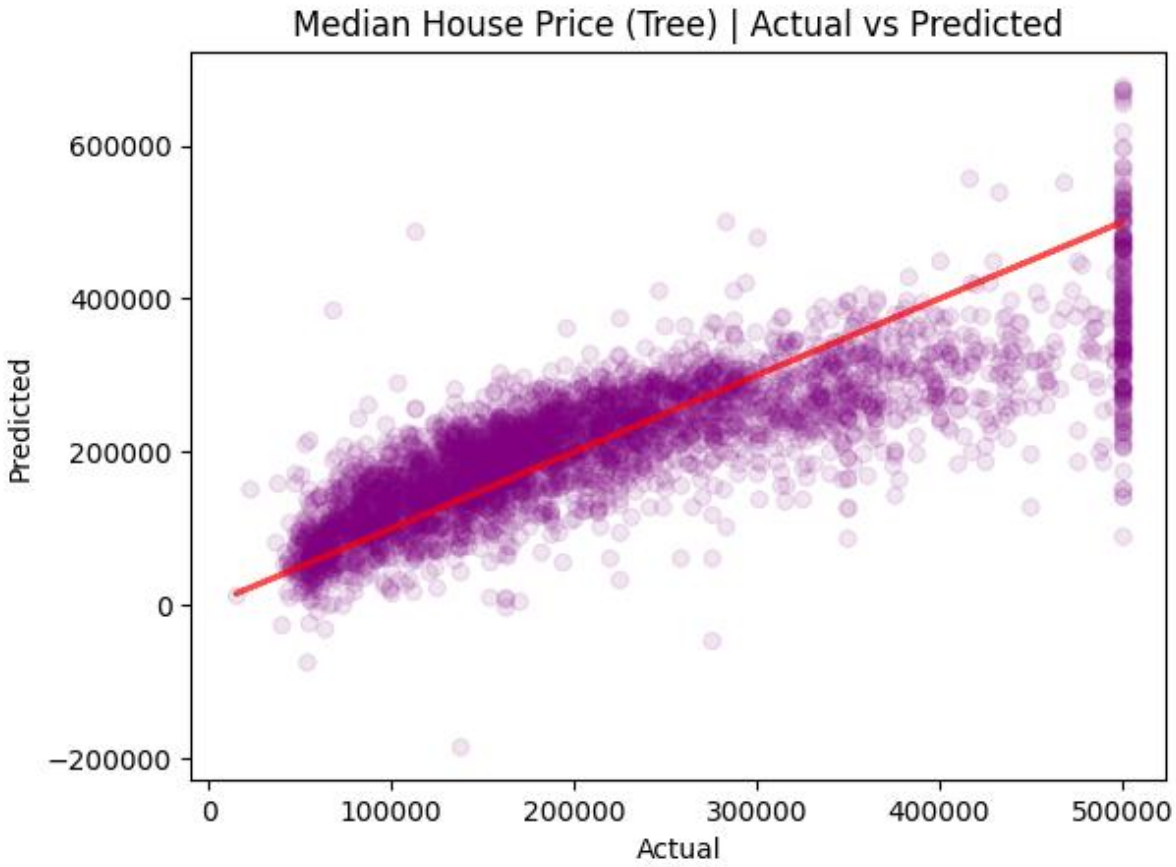
Median House Price (Tree) | Actual vs Predicted
Predicted
600000
400000
200000
0
−200000
0
100000
200000
300000
400000
500000
Actual

Figure 8A

```python
#Linear Regression

regressor = LinearRegression()
x_train, y_train = train_data.drop('median_house_value', axis=1), train_data['median_house_value']
regressor.fit(x_train, y_train)

print(f"Linear Regression Train R2 = {regressor.score(x_train, y_train)}")
print(f"Linear Regression Test R2 = {regressor.score(x_test, y_test)}")
```

✓ 0.0s

```
Linear Regression Train R2 = 0.6456482397646103
Linear Regression Test R2 = 0.6488402154431997
```

```python
#Random Forest Regressor
from sklearn.ensemble import RandomForestRegressor

forest = RandomForestRegressor()
forest.fit(x_train, y_train)

print(f"Random Forest Train R2 = {forest.score(x_train, y_train)}")
print(f"Random Forest Test R2 = {forest.score(x_test, y_test)}")
```

✓ 5.9s

```
Random Forest Train R2 = 0.9750209262157035
Random Forest Test R2 = 0.8256729590704017
```

Figure 9A

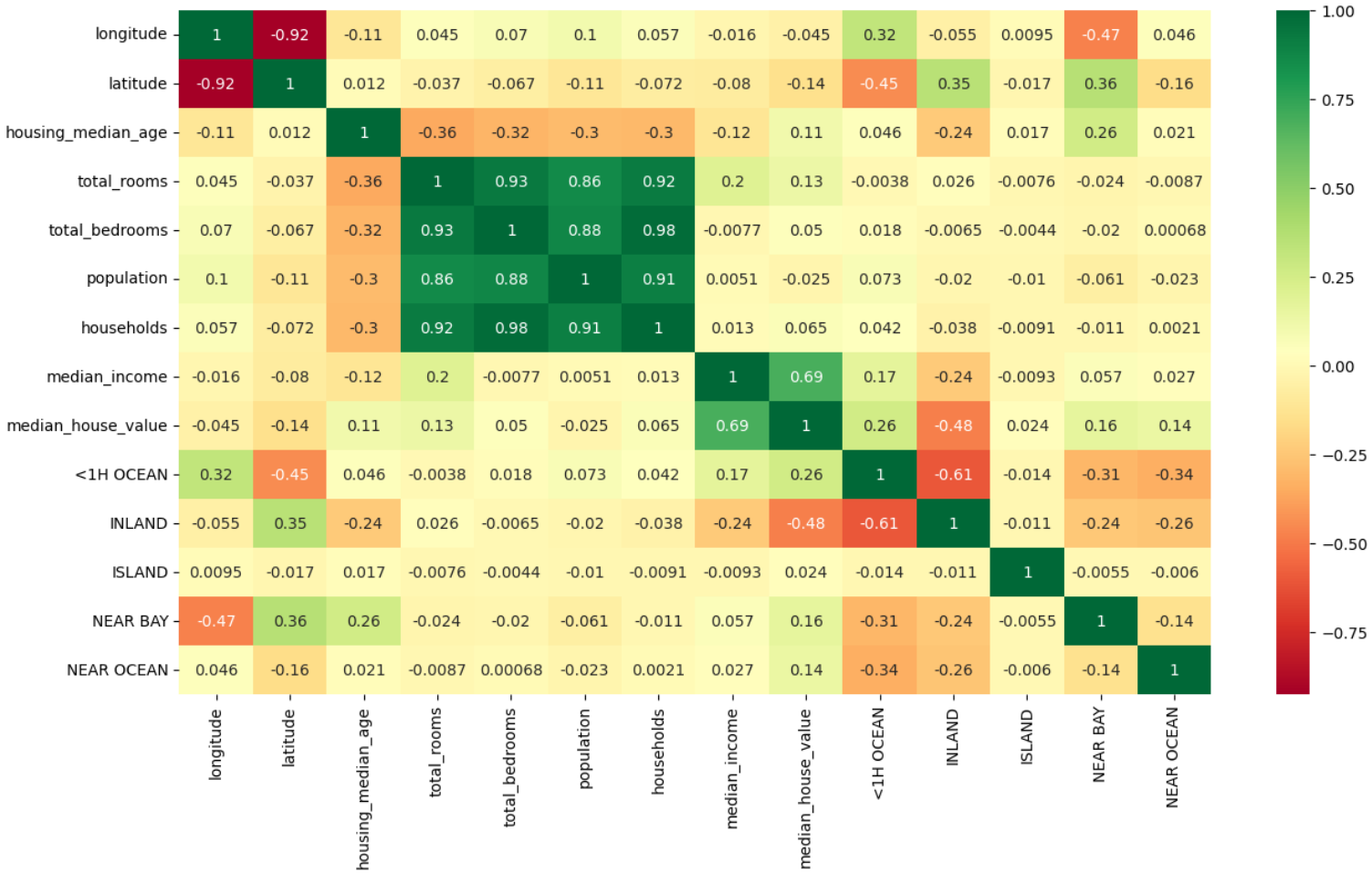
longitude 1 -0.92 -0.11 0.045 0.07 0.1 0.057 -0.016 -0.045 0.32 -0.055 0.0095 -0.47 0.046
latitude -0.92 1 0.012 -0.037 -0.067 -0.11 -0.072 -0.08 -0.14 -0.45 0.35 -0.017 0.36 -0.16
housing_median_age -0.11 0.012 1 -0.36 -0.32 -0.3 -0.3 -0.12 0.11 0.046 -0.24 0.017 0.26 0.021
total_rooms 0.045 -0.037 -0.36 1 0.93 0.86 0.92 0.2 0.13 -0.0038 0.026 -0.0076 -0.024 -0.0087
total_bedrooms 0.07 -0.067 -0.32 0.93 1 0.88 0.98 -0.0077 0.05 0.018 -0.0065 -0.0044 -0.02 0.00068
population 0.1 -0.11 -0.3 0.86 0.88 1 0.91 0.0051 -0.025 0.073 -0.02 -0.01 -0.061 -0.023
households 0.057 -0.072 -0.3 0.92 0.98 0.91 1 0.013 0.065 0.042 -0.038 -0.0091 -0.011 0.0021
median_income -0.016 -0.08 -0.12 0.2 -0.0077 0.0051 0.013 1 0.69 0.17 -0.24 -0.0093 0.057 0.027
median_house_value -0.045 -0.14 0.11 0.13 0.05 -0.025 0.065 0.69 1 0.26 -0.48 0.024 0.16 0.14
<1H OCEAN 0.32 -0.45 0.046 -0.0038 0.018 0.073 0.042 0.17 0.26 1 -0.61 -0.014 -0.31 -0.34
INLAND -0.055 0.35 -0.24 0.026 -0.0065 -0.02 -0.038 -0.24 -0.48 -0.61 1 -0.011 -0.24 -0.26
ISLAND 0.0095 -0.017 0.017 -0.0076 -0.0044 -0.01 -0.0091 -0.0093 0.024 -0.014 -0.011 1 -0.0055 -0.006
NEAR BAY -0.47 0.36 0.26 -0.024 -0.02 -0.061 -0.011 0.057 0.16 -0.31 -0.24 -0.0055 1 -0.14
NEAR OCEAN 0.046 -0.16 0.021 -0.0087 0.00068 -0.023 0.0021 0.027 0.14 -0.34 -0.26 -0.006 -0.14 1
longitude
latitude
housing_median_age
total_rooms
total_bedrooms
population
households
median_income
median_house_value
<1H OCEAN
INLAND
ISLAND
NEAR BAY
NEAR OCEAN
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75

Figure 10A

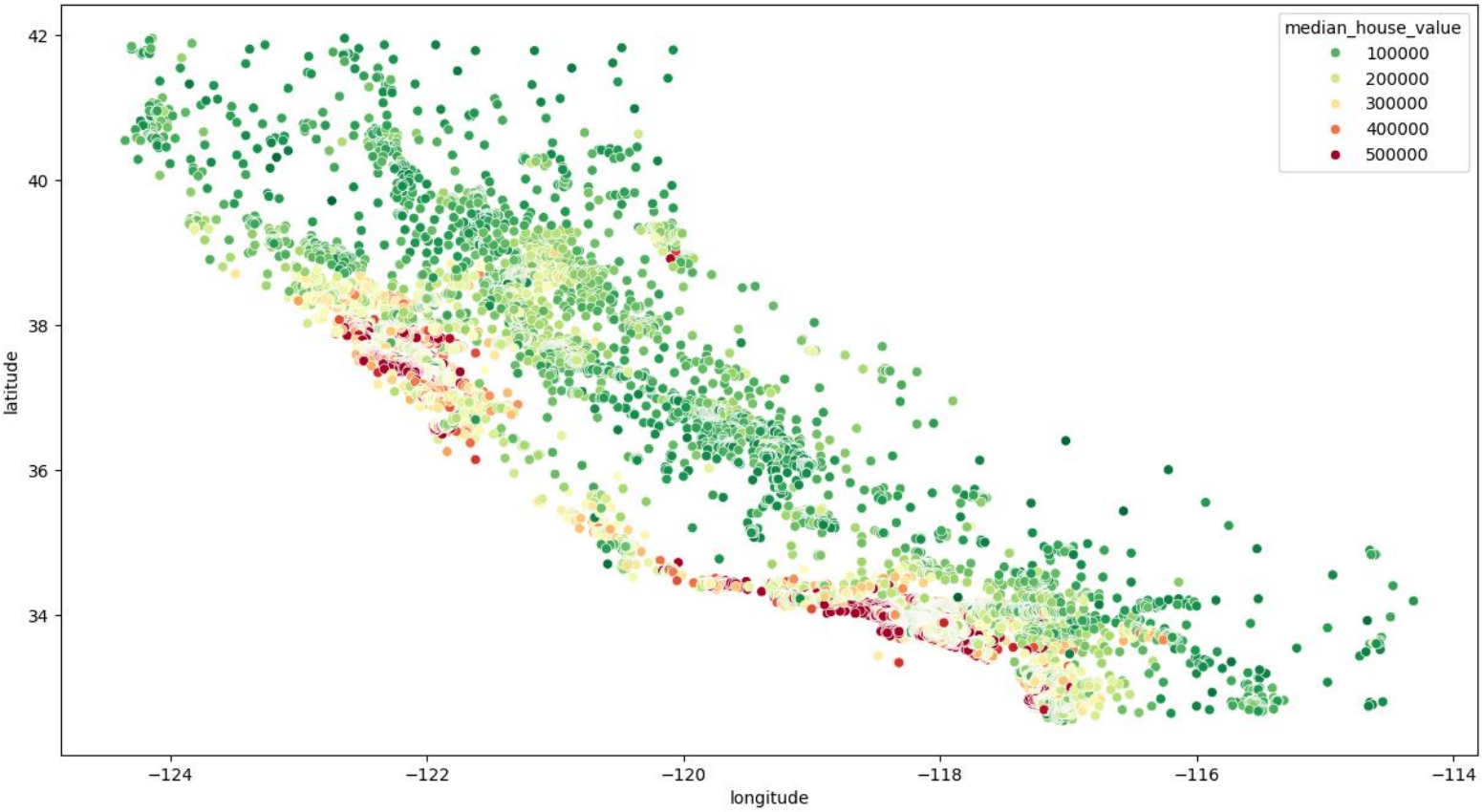
median_house_value
100000
200000
300000
400000
500000
latitude
42
40
38
36
34
longitude
−124
−122
−120
−118
−116
−114

Figure 11A

```python
#Linear House Prediction
linear_house_prediction = regressor.predict([[-122.23,#longitude
                                              37.88, #latitude
                                              41.0,  #housing_median_age
                                              880.0, #total_rooms
                                              129.0, #total_bedrooms
                                              322.0, #population
                                              126.0, #households
                                              8.3252,#median_income
                                              False, # 1< From Ocean
                                              False, #Inland
                                              False, #Island
                                              False, #Near Bay
                                              True]])#Near Ocean

#(GeeksforGeeks, 2025)
print(f"The median housing price of this block would be approximately ${linear_house_prediction[0]:.2f}")
```

✓ 0.0s

```
The median housing price of this block would be approximately $418238.61
c:\Users\camer\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarn
```

```python
#Forest House Prediction
forest_house_prediction = forest.predict([[-122.23,#longitude
                                           37.88, #latitude
                                           41.0,  #housing_median_age
                                           880.0, #total_rooms
                                           129.0, #total_bedrooms
                                           322.0, #population
                                           126.0, #households
                                           8.3252,#median_income
                                           False, # 1< From Ocean
                                           False, #Inland
                                           False, #Island
                                           False, #Near Bay
                                           True]])#Near Ocean

#(GeeksforGeeks, 2025)
print(f"The median housing price of this block would be approximately ${forest_house_prediction[0]:.2f}")
```

✓ 0.0s

```
The median housing price of this block would be approximately $455065.37
```

Figure 12A (Used Boards)

<table>
<tr><td>4x4 Easy 1</td><td><table><tr><td></td><td></td><td></td><td></td></tr><tr><td>1</td><td>3</td><td>2</td><td>4</td></tr><tr><td></td><td></td><td>4</td><td></td></tr><tr><td>2</td><td></td><td>3</td><td></td></tr></table></td></tr>
<tr><td>4x4 Easy 2</td><td><table><tr><td>4</td><td></td><td>1</td><td></td></tr><tr><td>1</td><td></td><td></td><td></td></tr><tr><td></td><td>1</td><td>4</td><td>3</td></tr><tr><td></td><td></td><td>2</td><td></td></tr></table></td></tr>
<tr><td>4x4 Easy 3</td><td><table><tr><td>2</td><td></td><td></td><td></td></tr><tr><td>4</td><td>3</td><td>2</td><td></td></tr><tr><td></td><td></td><td>1</td><td></td></tr><tr><td>1</td><td></td><td>3</td><td>4</td></tr></table></td></tr>
<tr><td>4x4 Medium 1</td><td></td></tr>
</table>

	3 | | 1 | | 2 | | | | | | | 1 | 2 | 4 | |
4x4 Medium 2	| | | 3 | 4 | | | | 3 | | 2 1 | | | 4
4x4 Medium 3	4 | 2 | 3 | | | | | | | | 1 | | | 1 | | 3
4x4 Hard 1	

	3 4 ·· ·· / ·· ·· 4 ·· / ·· 1 ·· ·· / ·· ·· ·· 1
4x4 Hard 2	·· ·· ·· 2 / 4 ·· ·· ·· / ·· 4 2 ·· / ·· ·· 3 ··
4x4 Hard 3	·· 4 3 ·· / ·· ·· ·· 4 / ·· ·· ·· ·· / 3 ·· ·· 2
4x4 Very Hard 1	

	3 2 1 4 4 1 3 2 1 4 2 3 2 3 4 1
4x4 Very Hard 2	1 3 4 2 4 2 1 3 2 1 3 4 3 4 2 1
4x4 Very Hard 3	4 2 3 1 3 1 2 4 2 4 1 3 1 3 4 2
6x6 Easy 1	

5	3		4		6
4	6		5	3	1
1	2	4		5	3
6		3			2
3	4		2	1	5
	1	5	3		4

6x6 Easy 2

	4	1			
3	2		6	4	1
5	6			1	
1	3	4	5	6	2
	1	6	4	3	5
4	5	3	1		6

6x6 Easy 3

5	4	1			2
2	6	3	4		5
3	1		6		4
6	2	4	5		1
1	5		2	4	3
			1	5	6

6x6 Medium 1	

		2	1		
	5	6		4	
5		4	3	1	6
6		3		2	
	4		6	5	2
2			4		

6x6 Medium 2

2	6	3	5		4
1		5			3
4	5	2	3		1
3		6	2		
			4		
	2	4			

6x6 Medium 3

	4		1		2
1		3		6	
	3		6	5	
6		5	4	2	3
2			3	1	
3					6

6x6 Hard 1

				5	
2	3	5		4	
			4	2	6
	4	2		3	
3		1	5		
		6		1	

6x6 Hard 2

2		4	3		
	3			5	
	1		6		3
3	6	2		4	5
		6			1
1					

6x6 Hard 3

	6	2	5		4
3		4	1	6	
6	4				
			6	4	
		6			1
5			4		

6x6 Very Hard 1

	3			1	6
6		1			
	5	2			
				5	3
				6	
		6			4

6x6 Very Hard 2

		4	1		
6	5				4
1		6			3
2	3		4	1	

6x6 Very Hard 3

2		5			3
		6			1
	3		2		
	6	4		1	
	2		6		

9x9 Easy 1

6		4	5			3	9	8
9			6				7	4
		8				6	2	1
				9		1		6
	1		4		3	9	5	7
	8	9			6	2	4	3
7		3		5		4	1	
				1				2
1	4				9	8		

9x9 Easy 2

2			4	6	7	5	3	
							1	
4		7		5	1	6		8
	1	8		3	2			
7	2	9		4	5		8	
	3					9	6	2
1	7	3		9	4	2		6
			5	2			9	
		2		1				3

9x9 Easy 3

4							2	9
7		6			9			
		5				7	3	8
	9	4	2				1	
		2		1			8	4
	7		5		4	3		
8		7		4		1	9	
2		1	9	5	3	8	7	6
	6				7	2	4	5

9x9 Medium 1

9								
6				1	7	3	4	2
			8	3				6
		4	6		3	1		
					2	5		
3	2				5	4	6	7
								4
7	6					2	1	5
						6		9

9x9 Medium 2

	1			5		9		
6		5						
2		3	1			5		
	3			6		4	9	2
				2				8
		8		7	3			
			6		7			
	5	9			4	8		3
	6		9			2	7	

9x9 Medium 3

7						5		8
			1	4		3		2
8			7					
	7	4			9		8	
2	3	8	6	7		9	5	
		1		3				
	8							
		6	3		5			
				6	7		3	1

9x9 Hard 1

4				7				
		7		9		4		1
	1			6				
5	4							
		8				5		2
9	2				5		8	4
			3		6		1	
3					8			
	5			2	4			7

9x9 Hard 2

6							8	
		1						
2						4		
	3			8	4	2		
5				9				6
		2		6	5			
					6	1		
		7	1		2			4
3		4						9

9x9 Hard 3

	3	7			5			9
					9			
		1				8		
4			3	7		2		
		5		6	4			
1						4	5	
9			1				2	6
		3	6			7		

9x9 Very Hard 1

		2						9
7			4					
		9		6			3	
2			7	8	4			
6			1					7
		4	2					
		3		2		8		
							5	
1			9		3	4		

9x9 Very Hard 2

	6	5	9			1	4	
				1	7		3	
		7						5
4								
5		6		7	9			
	9		1	4				
	5						1	
1				2			7	
	2		6					3

9x9 Very Hard 3

6			7					1
	3			2				9
			6	8		3		
			9			6		
5		6		1				4
				3				8
		7					1	
	9	2						
			1		7			6

Appendices

Appendix Part 1

```
import time

import tracemalloc

#Predefined Boards

Board9x9 = [

 [6, 0, 0, 7, 0, 0, 0, 0, 1],

 [0, 3, 0, 0, 2, 0, 0, 0, 9],

 [0, 0, 0, 6, 8, 0, 3, 0, 0],

 [0, 0, 0, 9, 0, 0, 6, 0, 0],

 [5, 0, 6, 0, 1, 0, 0, 0, 4],

 [0, 0, 0, 0, 3, 0, 0, 0, 8],

 [0, 0, 7, 0, 0, 0, 0, 1, 0],

 [0, 9, 2, 0, 0, 0, 0, 0, 0],

 [0, 0, 0, 1, 0, 7, 0, 0, 6]

]

Board6x6 = [

   [2, 0, 5, 0, 0, 3],

   [0, 0, 6, 0, 0, 1],

   [0, 3, 0, 2, 0, 0],

   [0, 0, 0, 0, 0, 0],

   [0, 6, 4, 0, 1, 0],

   [0, 2, 0, 6, 0, 0]

]

Board4x4 = [

   [4, 0, 0, 0],

   [0, 0, 0, 0],

   [0, 0, 0, 3],
```

```
    [1, 0, 4, 0]
]



#Board Size and Algorithm Selection.
algorithm = input("Choose solving algorithm (\"dfs\" or \"bfs\"): ").lower()
user_input = input("Enter Sudoku Board Size (\"4\", \"6\", or \"9\": ")
match user_input:
    case "4":
        Board, x, subgrid_width, y, RowDivider = (
        Board4x4, 2, 2, 3, "- - - - - -")

    case "6":
        Board, x, subgrid_width, y, RowDivider = (
        Board6x6, 2, 3, 5, "- - - - - - - - - ")

    case "9":
        Board, x, subgrid_width, y, RowDivider = (
        Board9x9, 3, 3, 8, "- - - - - - - - - - - - - " )

    case _:
        print("Invalid Input. Defaulting to 9x9 Board. Please enter ONLY \"4\" \"6\" or \"9\"")
        Board, x, subgrid_width, y, RowDivider = (
        Board9x9, 3, 3, 8, "- - - - - - - - - - - - - " )

#Human Readable Sudoku Output
def print_board(Board):
```

```
    for row in range(len(Board)):
        if row % x == 0 and row != 0:
            print(RowDivider)

        for column in range(len(Board[0])):
            if column % x == 0 and column != 0:
                print(" | ", end="")

            if column == y:
                print(Board[row][column])
            else:
                print(str(Board[row][column]) + " ", end="")
                #--- (TechWithTim, 2020)

#-----------------------------------------------------------
# Algorithm Functions

#Empty Cell Finder
def find_empty(Board):
    for row in range(len(Board)):
        for column in range(len(Board[0])):
            if Board[row][column] == 0:
                return (row, column)
    return None
            #--- (Coderivers, 2025)

#Valid move checker -----
def valid(board, row, col, number):
```

```
    if number in board[row]:
        return False

    if number in [board[i][col] for i in range(len(board))]:
        return False

    RowIndex1 = x * (row // x)
    ColumnIndex1 = subgrid_width * (col // subgrid_width)
    for i in range(RowIndex1, RowIndex1 + x):
        for j in range(ColumnIndex1, ColumnIndex1 + subgrid_width):
            if board[i][j] == number:
                return False
    return True
            #--- (TechWithTim, 2020)

#Depth-First Search Solver
def solve_dfs(Board):
    start_time = time.time()
    tracemalloc.start()

    valid_state = 0
    stack = [([row[:] for row in Board], 1)]

    while stack:
        board_peek, iterator = stack[-1]

        empty_cell = find_empty(board_peek)
        if empty_cell is None:
```

```
        for i in range(len(Board)):
            Board[i][:] = board_peek[i]

        x, peak = tracemalloc.get_traced_memory()
        tracemalloc.stop()
        end_time = time.time()
        print(f"Peak memory usage was {peak} bytes")
        print(f"Time Taken: {end_time - start_time:.6f} seconds")
        print(f"Solved | Valid States Checked {valid_state}")
        return True #Solved


    empty_row, empty_column = empty_cell
    if iterator > len(Board):
        stack.pop()
        continue
    stack.pop() == (board_peek, iterator + 1)
    stack.append((board_peek, iterator + 1))
    if valid(board_peek, empty_row, empty_column, iterator):
        valid_state += 1
        append_board = [row[:] for row in board_peek]
        append_board[empty_row][empty_column] = iterator
        stack.append((append_board, 1))

x, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()
end_time = time.time()
print(f"Peak memory usage was {peak} bytes")
```

```
    print(f"Time Taken: {end_time - start_time:.6f} seconds")
    print(f"Unsolved Valid States Checked {valid_state}")
    return False

def solve_bfs(Board):
    start_time = time.time()
    tracemalloc.start()

    valid_state = 0
    queue = [[row[:] for row in Board]]
    while queue:
        popboard = queue.pop(0)
        empty = find_empty(popboard)
        if empty == None:
            for i in range(len(Board)):
                Board[i][:] = popboard[i]

            x, peak = tracemalloc.get_traced_memory()
            tracemalloc.stop()
            end_time = time.time()
            print(f"Peak memory usage was {peak} bytes")
            print(f"Time Taken: {end_time - start_time:.6f} seconds")
            print(f"Solved | Valid States Checked {valid_state}")
            return True

        row1, column1 = empty
        for number in range(1, len(Board) + 1):
            valid_state += 1
```

```
        if valid(popboard, row1, column1, number):
            stackboard = [row[:] for row in popboard]
            stackboard[row1][column1] = number
            queue.append(stackboard)

    x, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    end_time = time.time()
    print(f"Peak memory usage was {peak} bytes")
    print(f"Time Taken: {end_time - start_time:.6f} seconds")
    print(f"Unsolved Valid States Checked {valid_state}")
    return False

if algorithm == 'dfs':
    if solve_dfs(Board):
        print_board(Board)
elif algorithm == 'bfs':
    if solve_bfs(Board):
        print_board(Board)
else:
    print("Invalid Algorithm Selection. Please enter ONLY \"dfs\" or \"bfs\"")



#-----------------------------------------------------------
#Code References
```

#TechWithTim (2020) Sudoku Solver Part 1. [online] Techwithtim.net. Available at: https://www.techwithtim.net/tutorials/python-programming/sudoku-solver-backtracking/part-1. [Accessed 10th Jan. 2026]

#Coderivers (2025). Sudoku Solver in Python: Unraveling the Logic and Best Practices. [online] Coderivers.org. Available at: https://coderivers.org/blog/sudoku-solver-python/ [Accessed 10th Jan. 2026]

Appendix Part 2

This code was run as a .ipynb in Juypter and may not function correctly within a .py file

```
# %%

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression


# %%

data = pd.read_csv('housing.csv')


# %%

data.dropna(inplace=True)

data = data.join(pd.get_dummies(data.ocean_proximity)).drop(['ocean_proximity'],
axis=1)

#(GeeksforGeeks, 2020)


# %%

standard_distribution = data.copy()
```

```
standard_distribution["total_rooms"] = np.log(data["total_rooms"])
standard_distribution["total_bedrooms"] = np.log(data["total_bedrooms"])
standard_distribution["population"] = np.log(data["population"])
standard_distribution["households"] = np.log(data["households"])
#   (Ryan & Matt Data Science, 2023)

# %%
standard_distribution.hist(figsize=(20,8))

# %%
plt.figure(figsize=(15,8))
sns.heatmap(data.corr(), annot=True, cmap="RdYlGn")

# %%
plt.figure(figsize=(15,8))
sns.scatterplot(x='longitude', y='latitude', hue='median_house_value', data=data,
palette='RdYlGn_r')

# %%
x = data.drop('median_house_value', axis=1)
y = data['median_house_value']

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
train_data = x_train.join(y_train)

# %%
#Linear Regression
```

```
regressor = LinearRegression()

x_train, y_train = train_data.drop('median_house_value', axis=1),
train_data['median_house_value']

regressor.fit(x_train, y_train)


print(f"Linear Regression Train R2 = {regressor.score(x_train, y_train)}")

print(f"Linear Regression Test R2 = {regressor.score(x_test, y_test)}")


# %%

#Random Forest Regressor

from sklearn.ensemble import RandomForestRegressor


forest = RandomForestRegressor()

forest.fit(x_train, y_train)


print(f"Random Forest Train R2 = {forest.score(x_train, y_train)}")

print(f"Random Forest Test R2 = {forest.score(x_test, y_test)}")


# %%

#Forest Scatter Graph


forest_pred = regressor.predict(x_test)

plt.scatter(y_test, forest_pred, color='purple', label='Predicted', alpha=0.1)

plt.plot(y_test, y_test, color='red', label='Actual', linewidth=2, alpha=0.7)

plt.xlabel("Actual")

plt.ylabel("Predicted")

plt.title("Median House Price (Tree) | Actual vs Predicted")
```

```
# %%
#Linear Scatter Graph

linear_pred = regressor.predict(x_test)
plt.scatter(y_test, linear_pred, color='blue', label='Predicted', alpha=0.1)
plt.plot(y_test, y_test, color='red', label='Actual', linewidth=2, alpha=0.7)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("Median House Price (Linear) | Actual vs Predicted")
#(GeeksforGeeks, 2024)

# %%
#Linear MAPE

from sklearn.metrics import mean_absolute_percentage_error

linear_MAPE = mean_absolute_percentage_error(y_test, linear_pred)
print(f"Linear - Mean Absolute Percentage Error (MAPE): {linear_MAPE:.2%}")

#Forest MAPE

forest_MAPE = mean_absolute_percentage_error(y_test, forest_pred)
print(f"Forest - Mean Absolute Percentage Error (MAPE): {forest_MAPE:.2%}")

#(scikit-learn, n.d.)

# %%
#Linear House Prediction
```

```
linear_house_prediction = regressor.predict([[-122.23,#longitude
                37.88, #latitude
                41.0,  #housing_median_age
                880.0, #total_rooms
                129.0, #total_bedrooms
                322.0, #population
                126.0, #households
                8.3252,#median_income
                False, # 1< From Ocean
                False, #Inland
                False, #Island
                False, #Near Bay
                True]])#Near Ocean

#(GeeksforGeeks, 2025)
print(f"The median housing price of this block would be approximately ${linear_house_prediction[0]:.2f}")

# %%
#Forest House Prediction
forest_house_prediction = forest.predict([[-122.23,#longitude
                37.88, #latitude
                41.0,  #housing_median_age
                880.0, #total_rooms
                129.0, #total_bedrooms
                322.0, #population
                126.0, #households
                8.3252,#median_income
```

```
                False, # 1< From Ocean
                False, #Inland
                False, #Island
                False, #Near Bay
                True]])#Near Ocean

#(GeeksforGeeks, 2025)
print(f"The median housing price of this block would be approximately ${forest_house_prediction[0]:.2f}")


# %%
#References ----------------------------------------------
```


#GeeksforGeeks (2024). Prediction Intervals for Machine Learning. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/machine-learning/prediction-intervals-for-machine-learning/ [Accessed 11 Jan. 2026].

#Ryan & Matt Data Science (2023). Data Science Beginner Project: Kaggle House Prices Regression Analysis (Full Walkthrough). [online] YouTube. Available at: https://www.youtube.com/watch?v=UqmulHG4IvY [Accessed 24 Aug. 2024].

#GeeksforGeeks (2025). Scikitlearn Cheatsheet [2025 Updated] Download pdf. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/blogs/scikit-learn-cheatsheet/ [Accessed 11 Jan. 2026].

#scikit-learn (n.d.). sklearn.metrics.mean_absolute_percentage_error. [online] Available at: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_absolute_percentage_error.html.[Accessed 11 Jan. 2026].

#GeeksforGeeks (2020). Pandas get_dummies() method. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/pandas/python-pandas-get_dummies-method/.[Accessed 11 Jan. 2026].

References

GeeksforGeeks (2019). *Random Forest Regression in Python*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/machine-learning/random-forest-regression-in-python/.

GeeksforGeeks (2021). *How to Calculate MAPE in Python?* [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python/how-to-calculate-mape-in-python/.

GeeksforGeeks (2020). *Python Coefficient of DeterminationR2 score*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/machine-learning/python-coefficient-of-determination-r2-score/.